

PSOC™ 4 safety test library

Customer training workshop (CTW)

July 2026

Public

Pre-requisites

- Basic hands-on with ModusToolbox™
- PSOC™ digital peripherals training
- Basic understanding of PSOC™ 4 devices
- **Install or update the following on your PC**
  - ModusToolbox™ Tools package v3.7.0
  - ModusToolbox™ Eclipse IDE
  - ModusToolbox™ Programming Tools 1.7
- **Hardware**
  - PSOC™ 4100T Plus Prototyping Kit - CY8CPROTO-041TP
  - MiniProg4 Programmer/debugger (CY8CKIT-005-A)
  - Jumper wires (4 qty)
  - A 150 kΩ and 75 kΩ through hole resistor
  - A breadboard

Copyright © Infineon Technologies AG 2026. All rights reserved.

Agenda

1	Introduction to functional safety standards	4
2	Introduction to Infineon's safety test library	7
3	Description of the core peripheral tests	11
4	Lab 1: Executing the core peripheral tests	25
5	Description of the analog peripheral tests	27
6	Lab 2: Executing the analog peripheral tests	32
7	Description of the communication block tests	34
8	Lab 3: Executing the UART loopback test	39

Copyright © Infineon Technologies AG 2025. All rights reserved.

Agenda

1	**Introduction to functional safety standards**	**4**
2	Introduction to Infineon's safety test library	7
3	Description of the core peripheral tests	11
4	Lab 1: Executing the core peripheral tests	25
5	Description of the analog peripheral tests	27
6	Lab 2: Executing the analog peripheral tests	32
7	Description of the communication block tests	34
8	Lab 3: Executing the UART loopback test	39

Copyright © Infineon Technologies AG 2025. All rights reserved.

Safety standards and compliance requirements

- Class B safety standards refer to a set of functional safety standards and guidelines that ensure the reliability and safety of microcontroller-based systems in safety-critical applications
- IEC 60730-1 overview: Safety standard for automatic electrical controls in household appliances (mechanical, electrical, EMC, abnormal operation)
- IEC 61508 SIL 2: Complementary industrial standard (functional safety of E/E/PE systems)

Class	Purpose / risk level	Typical examples	Software requirements	Library relevance
Class A	Not relied upon for safety; failure causes no hazard	Simple indicators, non-critical UI	Minimal/no diagnostics	Not applicable
Class B	Prevents unsafe operation; failure may lead to hazard but not immediate severe risk	Thermal protection, overcurrent in cookers/washers/refrigerators	Self-tests, fault detection, diagnostics (for example, CPU, memory, peripherals)	mtb-stl targets this – provides required self-tests for compliance
Class C	Prevents special hazards (for example, explosion/fire); failure can directly cause danger	Gas burner ignition/controls	Highest: redundancy, independent checks, advanced diagnostics	Higher than mtb-stl; often needs extra HW measures

Copyright © Infineon Technologies AG 2025. All rights reserved.

Safety standards and compliance requirements (contd.)

- Why focus on Class B?
  - Most household appliances (for example, induction cooktops, washing machines) fall into Class B. Infineon Class B libraries, such as mtb-stl, simplify certification without full redundancy
- Relevance to PSOC™ 4
  - Enables certified safety in low-power, sensor-based designs (for example, IoT, HMI boards in cooktops)
- Why use this library?
  - Automates fault detection, reduces development time, and ensures compliance for market certification

Copyright © Infineon Technologies AG 2025. All rights reserved.

Agenda

1	Introduction to functional safety standards	4
2	**Introduction to Infineon's safety test library**	**7**
3	Description of the core peripheral tests	11
4	Lab 1: Executing the core peripheral tests	25
5	Description of the analog peripheral tests	27
6	Lab 2: Executing the analog peripheral tests	32
7	Description of the communication block tests	34
8	Lab 3: Executing the UART loopback test	39

Copyright © Infineon Technologies AG 2025. All rights reserved.

Introduction to Infineon's Class B safety test library (mtb-stl library)

- How do we ensure Infineon products meet Class B standards?
  - We do have a library called mtb-stl that has self-diagnostic methods prescribed in the Class B standards
  - ModusToolbox™ Safety Test Library (mtb-stl) provides functional safety APIs for fault detection
  - Compliant with IEC 60730 Class B
  - The library is Scalable across Infineon MCUs, focused on PSOC™ 4 for this training
  - Implements self-tests and diagnostics for reliability in safety-critical environments (for example, appliances, industrial controls)
- For customers seeking certification for their safety-critical system, the mtb-stl Safety Software Library and Class-B certified Infineon devices will help customers achieve this faster
- The library is public and is available at https://github.com/Infineon/mtb-stl
- Code Examples that demonstrate the usage of the Safety Software Library by injecting an error - GitHub

Copyright © Infineon Technologies AG 2025. All rights reserved.

Class B (mtb-stl library) - Supported devices and tools

- **PSOC™ 4 families**
  - PSOC™ 4100S
  - PSOC™ 4100S Max
  - PSOC™ 4100S Plus 256 KB
  - PSOC™ 4500S
  - PSOC™ 4000S
  - PSOC™ 4000T
  - PSOC™ 4100S Plus
  - PSOC™ 4100T Plus
  - PSOC™ 4700S
- Designed for low-power, CAPSENSE™-enabled applications (buttons, sliders in appliances)
- PSOC™ 4 CAPSENSE™ middleware library is also Class B certified https://github.com/Infineon/capsense
- **Prerequisites**
  - ModusToolbox™ software environment
  - mtb-stl library added via library manager

Copyright © Infineon Technologies AG 2025. All rights reserved.

Self-test implementation in real-applications

- **Initial check**
  - All self-tests can be executed immediately after device startup
  - This enables verification that the device is suitable for operation before the application firmware is executed
- **Periodic check**
  - Self-tests executed during normal operation enable continuous fault detection and support user-defined corrective actions

Copyright © Infineon Technologies AG 2025. All rights reserved.

Agenda

1	Introduction to functional safety standards	4
2	Introduction to Infineon's safety test library	7
3	**Description of the core peripheral tests**	**11**
4	Lab 1: Executing the core peripheral tests	25
5	Description of the analog peripheral tests	27
6	Lab 2: Executing the analog peripheral tests	32
7	Description of the communication block tests	34
8	Lab 3: Executing the UART loopback test	39

Copyright © Infineon Technologies AG 2025. All rights reserved.

Tests performed for core peripherals

- Test for stuck-at faults using CPU registers – Stuck-at fault safety test
- Test for jumps to the correct address – Program counter safety test
- Test for proper program flow execution – Program execution flow safety test
- Test that interrupts are detected within a predefined range – Interrupt safety test
- Test for IMO clock reliability – Clock safety test
- Test for memory corruption – Flash safety test
- Test for transition fault and coupling fault – SRAM MARCH safety test
- Test stack overflow – Stack overflow safety test
- Test for pin shorts – GPIO safety test
- Test the WDT operation – Watchdog timer (WDT) safety test
- Test device configuration registers are correct – Config register safety test
- Test for valid DMA operation – DMA safety test

Copyright © Infineon Technologies AG 2026. All rights reserved.

Stuck-at fault safety test

- The PSOC™ 4 with Arm® Cortex®-M0 or Cortex®-M0+, CPU has 16-bit and 32-bit registers and they are as follows:
  - R0 to R12: General-purpose registers
  - R13: Stack pointer (SP)
  - R14: Link register
  - R15: Program counter
- A checkerboard test pattern is used to verify that bits in CPU registers R0–R14 are not stuck at logic ‘0’ or ‘1’
- The CPU register contents are saved on the stack before executing the test
- Each register is sequentially written as ‘01010101’ followed by ‘10101010’, and the values are read back for verification
- The Program Counter (R15) is the only CPU register not tested

Copyright © Infineon Technologies AG 2026. All rights reserved.

Program counter (PC) safety test

- **Standard**
  - Implements IEC 60730 H.2.16.5, which defines the functional test for the Program Counter
- **Program counter**
  - A CPU register that holds the address of the next instruction to execute
- **Test the technique Checkerboard** pattern on the PC value
  - Two test functions are placed at the addresses 0x5555 and 0xAAAA: bit-complementary patterns (0101… and 1010…)
  - These patterns exercise every PC bit in both 0 and 1 states, detecting stuck-at faults
- **Test sequence**
  - The CPU jumps to the function at 0x5555 → function returns a known signature
  - The CPU jumps to the function at 0xAAAA → function returns a known signature
  - Confirm that the CPU reaches each address and returns correctly to the calling function
- **Pass/Fail**
  - If both signatures match, the function returns the expected values, indicating that the PC is functioning correctly
  - If a signature mismatch, improper return, or hard fault is detected, a PC error shall be reported, and the system shall transition to the safe state

Note: The linker script must reserve and place the two test stubs at exactly 0x5555 and 0xAAAA in flash memory.

Copyright © Infineon Technologies AG 2025. All rights reserved.

Program execution flow safety test

- **Goal**
  - Ensure that each critical code block is entered and exited through the intended execution path and in the correct sequence
- **Mechanism**
  - Maintain a pair of **complementary counters** (for example, one incrementing, and the other decrementing)
- **At each critical block**
  - **Before** execution → add a **unique value** to counter A and subtract the same value from counter B
  - **After** execution → reverse the operation (subtract from A, add to B)
- **Invariant**
  - If entry points is equals exit points, the counter pair must remain **complementary** (A + B ==constant) after each block
- **Fault detection**
  - Block **skipped** → entry/exit mismatch → counters non-complementary
  - Block **executed multiple times** or out of order → unique values do not cancel → mismatch occurs
  - Any unexpected counter value → **program flow execution error** → System transitions to a safe state

Copyright © Infineon Technologies AG 2026. All rights reserved.

Interrupt safety test

- In Arm® Cortex®-M devices, the nested vectored interrupt controller (NVIC) enables hardware events to redirect the CPU from main code to an ISR, and then resumes the interrupted code
- Implements independent time-slot monitoring defined in IEC 60730 H.2.18.10.4
- **Goal**
  - Confirm interrupts occur regularly and at the expected rate
  - Validate the correct operation of the NVIC together with a timer-driven interrupt source
- **Method**
  - The function SelfTest_Interrupt() is called to initiate the test by starting the reference timers
  - A timer generates a periodic interrupt, and the corresponding ISR increments a counter
  - After the time window, the counter value is compared against predefined min/max limits
- **PSOC™ 4 implementation**
  - Timer_1 is configured to generate 13 interrupts per 1 ms
  - isr_1 counts the number of interrupts that occurred
  - Pass condition: count ≥ 9 and ≤ 15 , test passed
  - An out-of-range count indicates an interrupt fault and triggers a transition to the safe state

Copyright © Infineon Technologies AG 2026. All rights reserved.

Clock safety test

- This safety test verifies the reliability of the internal main oscillator (IMO) system clock
  - It confirms that the clock is within the tolerance of the internal low-speed oscillator (ILO). The 32 kHz ILO clock is accurate to ± 10%
- **Timer 0**
  - Input frequency = 100x10^6
  - Period = 10^5
  - Delay = 1 ms
- **WDT**
  - Input frequency = 32 kHz (ILO)
  - Get the initial count of WDT
  - Start counter for 1ms
  - Get WDT final count in Timer 0 ISR
  - 20 <(Final count – initial count = 32) <80 => success

Copyright © Infineon Technologies AG 2025. All rights reserved.

Flash safety test

- **Goal**
  - Provide a comprehensive diagnostic of the program flash by computing a checksum over all used flash and comparing it against a stored reference
- **Checksum method**
  - The library supports Fletcher 64-bit or CRC-32.
  - Selectable at build time via the FLASH_TEST_MODE macro
    - **FLASH_TEST_FLETCHER64**: Fletcher 64-bit
    - **FLASH_TEST_CRC32**: CRC-32
- **How it works**
  - The test reads every ROM / flash location sequentially
  - Each value is accumulated into a 64-bit running checksum
  - The traversal stops at end-of-flash minus 8 bytes
  - The reference 64-bit checksum is stored in the last 8 bytes of the flash itself
  - A custom linker script places this reference checksum at the correct fixed location
- **Pass/fail**
  - Calculated checksum matches the stored reference, PASS (Flash ok)
  - Checksum mismatch; the flash failure is detected, FAIL (code execution is frozen to prevent running invalid code and enter safe state)

Note: The reference checksum is patched in by a post-build step after the firmware is linked.

Copyright © Infineon Technologies AG 2025. All rights reserved.

SRAM (March) safety test

- Stuck-at fault, transition fault and coupling fault are detected:
  - Transition fault - When a signal or logic value fails to transition correctly between states (for example, from 0 to 1 or from 1 to 0) within the expected time frame or under specific conditions
  - Coupling fault - The behaviour of one memory cell or logic element unintentionally affects another due to unintended interactions or dependencies. The memory cells that are physically close to each other may interfere due to capacitive or inductive coupling

- **Test sequence**

1. This test first duplicates data in the area to be tested, as this test is destructive
2. Write 0 to the block in ascending order
3. Read 0 and write 1 in ascending order
4. Read 1 and write 0 in ascending order
5. Read 0
6. Read 0 and write 1 in descending order
7. Read 1 and write 0 in descending order
8. Read 0
9. Copy back the original data function

Copyright © Infineon Technologies AG 2025. All rights reserved.

Stack overflow safety test

- **Stack**
  - A region of RAM that the CPU uses for temporary storage of data and addresses, since the register file is limited
  - PSOC™ 4 stack location: The stack is placed at the end of RAM and grows downward
- **Goal**
  - Ensure that the stack does not overlap with program data memory during execution
  - Detect runaway growth caused by deep call chains, recursion, or interrupt nesting
- **Method**
  - A fixed memory block is reserved at the end of the stack region (the guard area)
  - This block is filled with a predefined guard pattern
  - Periodically call the self-test function to read back the guard area and verify that the pattern remains intact
- **Pass/fail**
  - Pattern is unchanged, stack remains within bounds, and test is passed
  - If any byte in the reserved block is corrupted, a stack overflow is detected, and the system enters a safe state

Note: The same guard-pattern technique can be applied at the opposite end to detect stack underflow.

Copyright © Infineon Technologies AG 2025. All rights reserved.

GPIO safety test

- **Goal**
  - Verify that GPIO pins are not shorted to GND or VCC
- **Principle**
  - Under normal conditions, pin-to-GND and pin-to-VCC resistances are very HIGH
  - The internal pull-up/pull-down resistor is used as a known reference impedance
  - Comparing the read level against the expected level reveals an external short-circuit
- **Detect pin-to-GND short**
  - Configure the pin in resistive pull-up drive mode
  - Normal condition: The pull-up pulls the line HIGH; CPU reads logic 1
  - Fault condition: If the pin is shorted to GND (low resistance), the input is pulled LOW, the CPU reads logic 0 and a fault is detected
- **Detect pin-to-VCC short**
  - Configure the pin in resistive pull-down drive mode
  - Normal condition: the pull-down pulls the line LOW; CPU reads logic 0
  - Fault condition: If the pin is shorted to VCC, the input is pulled HIGH, the CPU reads logic 1 and a fault is detected
- **Pass/fail**
  - Read level matches the expected level for the configured pull mode → GPIO test passed
  - Read level differs from expected → short to GND or VCC detected→ enter safe state

Copyright © Infineon Technologies AG 2025. All rights reserved.

Watchdog timer (WDT) safety test

- **Goal**
  - Confirm the watchdog actually generates a reset when it is not serviced, i.e., the WDT is alive and effective
- **Method**
  - The test function initialises the WDT and then enters an infinite loop without servicing it
  - If the WDT is operating correctly, the timeout expires, and the device is reset
  - After the reset, the function analyzes the reset source register on startup
    - If the reset source is the watchdog, the WDT is functional, and the function returns OK
    - If the reset source indicates any other reset cause, an infinite loop continues to execute (test does not pass)
- **Pass/Fail**
  - Reset cause = WDT after the forced loop; the test passed
  - If there is no reset, or the reset is from a different source, WDT fault and enter safe state

Copyright © Infineon Technologies AG 2026. All rights reserved.

Config register safety test

- Verify that the static startup configuration registers retain their intended values throughout runtime. Detect unintended changes caused by hardware faults, software bugs, or peripheral misconfiguration
- Startup configuration registers are typically static and are configured in files prefixed with “cycfg” (for example, cycfg_clocks.c)
- **Static vs dynamic registers**
  - Test applies only to static registers, values fixed after startup
  - Registers that are legitimately updated at runtime must be excluded from this test
  - Such dynamic registers are instead validated by the application, which knows the current correct value
- **Two supported test modes**
  - **Mode 1 — Duplicate-in-flash compare**
    - After device startup, copies of the startup configuration registers are stored in flash
    - Periodically, the live registers are compared against these stored duplicates
    - On a mismatch, the corrupted register can be restored from the flash copy
  - **Mode 2 — CRC compare with the status semaphore**
    - If the CRC status semaphore is set, compute the live CRC over the register set and compare against the CRC previously stored in flash
    - If the semaphore is not set, compute the CRC, store it in flash, and set the status semaphore (initial baseline)
- **Pass/fail**
  - The live values (or computed CRC) match the stored reference; configuration intact
  - A mismatch indicates configuration drift has been detected. Restore the configuration from the stored copy and/or enter safe state

Copyright © Infineon Technologies AG 2026. All rights reserved.

DMA safety test

- Verify that the DMA controller transfers data correctly (source addressing, destination addressing, transfer width, and increment behavior
- **Test sequence**
  - **Step 1: Clear destination**
    - A 64-byte destination block is cleared to ‘0’
    - Performed using DMA transfers consisting of 16 × 32-bit writes from a fixed source address.
  - **Step 2: Verify clear**
    - Read back the destination block and confirm every byte is ‘0’
  - **Step 3: Pattern fill**
    - Fill the same destination block with the repeating pattern 00 00 FF
    - Performed using an 8-bit DMA from a fixed source address
    - Destination address increment = 1
    - Transfer length = 22
  - **Step 4: Verify pattern**
    - Read back the destination block and confirm the expected byte sequence (lowest address first): FF 00 00 FF 00 00 FF 00 00 FF 00 00 FF 00 00 FF 00 00 FF 00 00 FF 00 00 …
- **Pass/fail**
  - Both verifications match the expected contents. DMA test has been passed
  - Any mismatch: there is a DMA fault; enter safe state

Copyright © Infineon Technologies AG 2026. All rights reserved.

Agenda

1	Introduction to functional safety standards	4
2	Introduction to Infineon's safety test library	7
3	Description of the core peripheral tests	11
4	**Lab 1: Executing the core peripheral tests**	**25**
5	Description of the analog peripheral tests	27
6	Lab 2: Executing the analog peripheral tests	32
7	Description of the communication block tests	34
8	Lab 3: Executing the UART loopback test	39

Copyright © Infineon Technologies AG 2025. All rights reserved.

Lab 1: Executing the core peripheral tests

Agenda

1	Introduction to functional safety standards	4
2	Introduction to Infineon's safety test library	7
3	Description of the core peripheral tests	11
4	Lab 1: Executing the core peripheral tests	25
5	**Description of the analog peripheral tests**	**27**
6	Lab 2: Executing the analog peripheral tests	32
7	Description of the communication block tests	34
8	Lab 3: Executing the UART loopback test	39

Copyright © Infineon Technologies AG 2025. All rights reserved.

Tests performed for analog peripherals

- Test that the ADC is operational within limits and to specified accuracy – ADC safety test
- Test that the comparator is operating using internal and external references – Comparator safety test
- Test that the op-amp is operating at the specified tolerances – Op-amp safety test

Copyright © Infineon Technologies AG 2026. All rights reserved.

ADC safety test

- Implements the independent input comparison defined in IEC 60730 H.2.18.8
  - Compares the result against a known reference with tolerances
- **Goal**
  - Verify correct operation of the SAR ADC analog functions, including channel routing, reference, and conversion)
- **Method**
  - Each ADC channel that is enabled for the test is connected to a known reference voltage
  - The ADC converts the reference, and then the resulting code is compared against an expected value
- **Pass/fail**
  - The measured value is within the expected min/max range, and the ADC test is passed
  - The measured value is not within the expected range; an ADC fault is detected. Returns an error and enters a safe state

Copyright © Infineon Technologies AG 2025. All rights reserved.

Comparator safety test

- **Goal**
  - Verify that each analog comparator correctly resolves its positive (+ve) and negative (−ve) inputs and produces the expected digital output
- **Method A** — Inputs routable via the programmable analog block
  - Connect one comparator to the internal reference voltage and the other input to ground
  - Read the comparator output and verify that it matches the expected logic level
  - Swap the connections (reference ↔ ground) on the two inputs
  - Read the output again and check that it changes to the opposite of the expected logic level
- **Method B** — Inputs not routable through the analog block
  - Use the GPIO internal pull-up and pull-down resistors to force known levels on the comparator inputs
  - Apply pull-up on one input and pull-down on the other → verify the expected output
  - Reverse the pull-up and pull-down configuration and verify that the output changes to the opposite logic level
- **Pass/fail**
  - Comparator output matches the expected level in both input configurations. The comparator test is passed
  - Wrong or unchanged output when inputs are swapped. A comparator fault is detected and enters a safe state

Copyright © Infineon Technologies AG 2025. All rights reserved.

Op-amp safety test

- **Goal**
  - Verify the correct operation of the on-chip operational amplifier (op-amp) signal path
- **Method**
  - Configure the op-amp as a unity-gain buffer (output = input)
  - Apply a known input voltage to the op-amp
  - Route the op-amp output to the SAR ADC and convert
  - Compare the measured ADC code against the expected value (matching the applied input within a defined tolerance)
- **Pass/fail**
  - The measured output matches the applied input within the specified tolerance. The op-amp test is passed
  - The measured output deviates from the expected value beyond tolerance. An op-amp fault is detected, and the op-amp enters a safe state

Copyright © Infineon Technologies AG 2025. All rights reserved.

Agenda

1	Introduction to functional safety standards	4
2	Introduction to Infineon's safety test library	7
3	Description of the core peripheral tests	11
4	Lab 1: Executing the core peripheral tests	25
5	Description of the analog peripheral tests	27
6	**Lab 2: Executing the analog peripheral tests**	**32**
7	Description of the communication block tests	34
8	Lab 3: Executing the UART loopback test	39

Copyright © Infineon Technologies AG 2025. All rights reserved.

Lab 2: Executing the analog peripheral tests

Agenda

1	Introduction to functional safety standards	4
2	Introduction to Infineon's safety test library	7
3	Description of the core peripheral tests	11
4	Lab 1: Executing the core peripheral tests	25
5	Description of the analog peripheral tests	27
6	Lab 2: Executing the analog peripheral tests	32
7	**Description of the communication block tests**	**34**
8	Lab 3: Executing the UART loopback test	39

Copyright © Infineon Technologies AG 2025. All rights reserved.

Tests performed for communication blocks

- Test for UART data transmission and reception – UART safety test
- Test for SPI data transmission and reception – SPI safety test
- Test for I2C data transmission and reception in master mode – I2C Master safety test

Copyright © Infineon Technologies AG 2026. All rights reserved.

UART safety test

- **Goal**
  - Verify that the UART transmit and receive paths, including FIFO cells, work correctly using internal loopback
- **Mechanism**
  - The Serial Communication Block (SCB) has a built-in internal loopback that connects TX to RX, or loopback using Smart I/O
  - This enables the test to run without any external wiring
- **Method**
  - On each call, the function transmits one test byte and reads the byte received through the loopback
  - Compare the transmitted byte to the received byte
  - The test byte is incremented on every call
  - The test runs across all 256-byte values to exercise every cell of the 256-byte FIFO
- **Pass/fail**
  - Return code 3: Overall test passed
  - Return code 1, 4, 5, or 6: UART fault is detected and enters a safe state
    - Error code 1: Test failed (TX byte != RX byte)
    - Error code 4: TX FIFO not empty at start of test
    - Error code 5: RX FIFO not empty at start of test
    - Error code 6: UART is not enabled
  - Return code 2: In progress; continue calling the function until a pass or fail result is returned

Copyright © Infineon Technologies AG 2025. All rights reserved.

SPI safety test

- **Goal**
  - Verify that the SPI transmit and receive paths, including FIFO cells, work correctly using internal loopback
- **Mechanism**
  - The Serial Communication Block (SCB) has a built-in internal loopback that connects MOSI to MISO, or uses Smart I/O
  - This enables the test to run without any external wiring
- **Method**
  - On each call, the function transmits one test byte and reads the byte received through the loopback
  - Compare the transmitted byte to the received byte
  - The test byte is incremented on every call
  - The test runs across all 256-byte values to exercise every cell of the 256-byte FIFO
- **Pass/fail**
  - Return code 3: Overall test passed
  - Return code 1, 4, or 5: UART fault is detected and enters a safe state
    - Error code 1: Test failed (TX byte != RX byte)
    - Error code 4: TX FIFO not empty at start of test
    - Error code 5: RX FIFO not empty at start of test
  - Return code 2: In progress; continue calling the function until a pass or fail result is returned

Copyright © Infineon Technologies AG 2025. All rights reserved.

I2C master safety test

- **Goal**
  - Verify correct end-to-end operation of the I2C bus, including the master, slave, addressing, write path, read path, and data integrity
- **Setup**
  - Connect the I2C master and slave externally through SDA, SCL, and common ground
  - Configure the slave with address 0x08
- **Method**
  - The master periodically transmits I2C data to the slave at address 0x08
  - The slave receives the data, computes its bitwise complement, and writes the result into its read buffer
  - The master then performs a read from the slave to retrieve the buffer contents
  - The master compares the received bytes against the complement of the data it sent in the previous transaction
- **Pass/fail**
  - Return code 3: I2C test passed
  - Return code 1: An I2C fault is detected, and enters a safe state
  - Return code 2: In progress; continue calling the function until 1 or 3 is returned

Copyright © Infineon Technologies AG 2025. All rights reserved.

Agenda

1	Introduction to functional safety standards	4
2	Introduction to Infineon's safety test library	7
3	Description of the core peripheral tests	11
4	Lab 1: Executing the core peripheral tests	25
5	Description of the analog peripheral tests	27
6	Lab 2: Executing the analog peripheral tests	32
7	Description of the communication block tests	34
8	**Lab 3: Executing the UART loopback test**	**39**

Copyright © Infineon Technologies AG 2025. All rights reserved.

Lab 3: Executing the UART loopback test

Revision history

Document revision	Date	Description of change
**	2026-07-24	Initial release.

Copyright © Infineon Technologies AG 2025. All rights reserved.



Thank you!
Stay tuned for more trainings…

Join with us on our mission to drive decarbonization and digitalization Together.

Public